3、高级面试题

由 涤生老师-健哥创建, 最后修改于一月 04, 2025

- 1、Kafka中的事务是怎么实现的？（理解）
- 2、失效副本是指什么？有那些应对措施？（理解）
- 3、多副本下，各个副本中的HW和LEO的演变过程（理解）
- 4、Kafka是如何保障数据不丢失的？（理解背诵）
- 5、如何解决Kafka数据丢失问题？（理解背诵）
- 6、Kafka可以保障永久不丢失数据吗？（理解）
- 7、Kafka在可靠性方面做了哪些改进？（理解）
- 8、如何保障Kafka中的消息是有序的？（理解背诵）
- 9、如何确定合适的Kafka主题的分区数量？（理解）
- 10、为什么Kafka不支持读写分离？（了解）
- 11、Kafka中的延迟队列怎么实现（理解）
- 12、Kafka中怎么实现死信队列和重试队列？（理解）
- 13、Kafka中怎么做消息审计？（了解）
- 14、Kafka中怎么做消息轨迹？（了解）
- 15、怎么计算Lag？(注意read_uncommitted和read_committed状态下的不同)（理解）
- 16、Kafka有哪些指标需要着重关注？（了解）
- 17、Kafka如何实现高吞吐的原理？（理解背诵）
- 18、Kafka里面存的数据格式是什么样的？（了解）
- 19、在实际的生产中，kafka常见的监控实现有哪些？（了解）
- 20、Kafkag高低级（新旧）API区别（了解）
- 21、Kafka在哪些地方会有选举过程，使用什么工具支持选举？（理解背诵）

1、Kafka中的事务是怎么实现的？（理解）

Kafka中的事务可以使应用程序将消费消息、生产消息、提交消费位移当作原子操作来处理，同时成功或失败，即使该生产或消费会跨多个分区。

生产者必须提供唯一的transactionalId，启动后请求事务协调器获取一个PID，transactionalId与PID一一对应。

每次发送数据给<Topic, Partition>前，需要先向事务协调器发送AddPartitionsToTxnRequest，**事务协调器会将该<Transaction, Topic, Partition>存于__transaction_state内，并将其状态置为BEGIN。**

在处理完 AddOffsetsToTxnRequest 之后，生产者还会发送 TxnOffsetCommitRequest 请求给 GroupCoordinator，从而将本次事务中包含的消费位移信息 offsets 存储到主题 __consumer_offsets 中

一旦上述数据写入操作完成，应用程序必须调用KafkaProducer的commitTransaction方法或者abortTransaction方法以结束当前事务。无论调用 commitTransaction() 方法还是 abortTransaction() 方法，生产者都会向 TransactionCoordinator 发送 EndTxnRequest 请求。
TransactionCoordinator 在收到 EndTxnRequest 请求后会执行如下操作：

1. 将 PREPARE_COMMIT 或 PREPARE_ABORT 消息写入主题 __transaction_state
2. 通过 WriteTxnMarkersRequest 请求将 COMMIT 或 ABORT 信息写入用户所使用的普通主题和 __consumer_offsets
3. 将 COMPLETE_COMMIT 或 COMPLETE_ABORT 信息写入内部主题 __transaction_state标明该事务结束

在消费端有一个参数isolation.level，设置为“read_committed”，表示消费端应用不可以看到尚未提交的事务内的消息。如果生产者开启事务并向某个分区值发送3条消息 msg1、msg2 和 msg3，在执行 commitTransaction() 或 abortTransaction() 方法前，设置为“read_committed”的消费端应用是消费不到这些消息的，不过在 KafkaConsumer 内部会缓存这些消息，直到生产者执行 commitTransaction() 方法之后它才能将这些消息推送给消费端应用。反之，如果生产者执行了 abortTransaction() 方法，那么 KafkaConsumer 会将这些缓存的消息丢弃而不推送给消费端应用。

2、失效副本是指什么？有那些应对措施？（理解）

正常情况下，分区的所有副本都处于 ISR 集合中，但是难免会有异常情况发生，从而某些副本被剥离出 ISR 集合中。在 ISR 集合之外，也就是处于同步失效或功能失效（比如副本处于非存活状态）的副本统称为**失效副本**，失效副本对应的分区也就称为同步失效分区，即 under-replicated 分区。

Kafka 从 0.9.x 版本开始就通过唯一的 broker 端参数 replica.lag.time.max.ms 来抉择，**当 ISR 集合中的一个 follower 副本滞后 leader 副本的时间超过此参数指定的值时则判定为同步失败，需要将此 follower 副本剔除出 ISR 集合。**replica.lag.time.max.ms 参数的默认值为10000。

在 0.9.x 版本之前，Kafka 中还有另一个参数 replica.lag.max.messages（默认值为4000），它也是用来判定失效副本的，当一个 follower 副本滞后 leader 副本的消息数超过 replica.lag.max.messages 的大小时，则判定它处于同步失效的状态。它与 replica.lag.time.max.ms 参数判定出的失效副本取并集组成一个失效副本的集合，从而进一步剥离出分区的 ISR 集合。

Kafka 源码注释中说明了一般有这几种情况会导致副本失效：

- follower 副本进程卡住，在一段时间内根本没有向 leader 副本发起同步请求，比如频繁的 Full GC。
- follower 副本进程同步过慢，在一段时间内都无法追赶上 leader 副本，比如 I/O 开销过大。
- 如果通过工具增加了副本因子，那么新增加的副本在赶上 leader 副本之前也都是处于失效状态的。

- 如果一个 follower 副本由于某些原因（比如宕机）而下线，之后又上线，在追赶上 leader 副本之前也处于失效状态。

应对措施

我们用UnderReplicatedPartitions代表leader副本在当前Broker上且具有失效副本的分区的个数。

如果集群中有多个Broker的UnderReplicatedPartitions保持一个大于0的稳定值时，一般暗示着集群中有Broker已经处于下线状态。这种情况下，这个Broker中的分区个数与集群中的所有UnderReplicatedPartitions（处于下线的Broker是不会上报任何指标值的）之和是相等的。通常这类问题是由于机器硬件原因引起的，但也有可能是由于操作系统或者JVM引起的 。

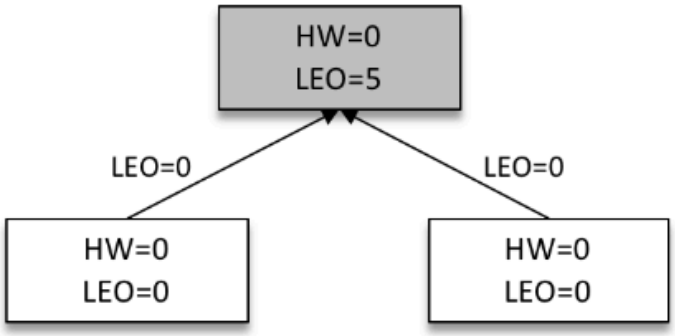
如果集群中存在Broker的UnderReplicatedPartitions频繁变动，或者处于一个稳定的大于0的值（这里特指没有Broker下线的情况）时，一般暗示着集群出现了性能问题，通常这类问题很难诊断，不过我们可以一步一步的将问题的范围缩小，比如先尝试确定这个性能问题是否只存在于集群的某个Broker中，还是整个集群之上。如果确定集群中所有的under-replicated分区都是在单个Broker上，那么可以看出这个Broker出现了问题，进而可以针对这单一的Broker做专项调查，比如：操作系统、GC、网络状态或者磁盘状态（比如：iowait、ioutil等指标）。

3、多副本下，各个副本中的HW和LEO的演变过程（理解）

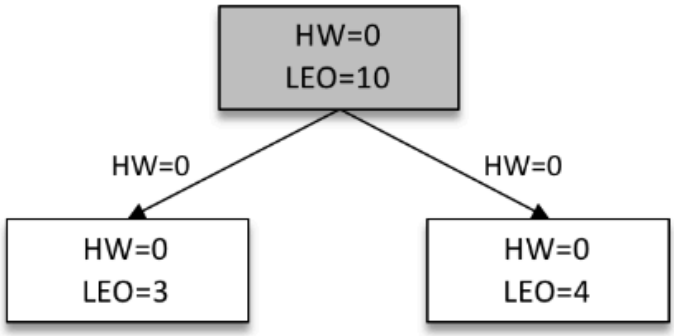
某个分区有3个副本分别位于 broker0、broker1 和 broker2 节点中，假设 broker0 上的副本1为当前分区的 leader 副本，那么副本2和副本3就是 follower 副本，整个消息追加的过程可以概括如下：

1. 生产者客户端发送消息至 leader 副本（副本1）中。
2. 消息被追加到 leader 副本的本地日志，并且会更新日志的偏移量。
3. follower 副本（副本2和副本3）向 leader 副本请求同步数据。
4. leader 副本所在的服务器读取本地日志，并更新对应拉取的 follower 副本的信息。
5. leader 副本所在的服务器将拉取结果返回给 follower 副本。
6. follower 副本收到 leader 副本返回的拉取结果，将消息追加到本地日志中，并更新日志的偏移量信息。

某一时刻，leader 副本的 LEO 增加至5，并且所有副本的 HW 还都为0。



情形1（初识状态）

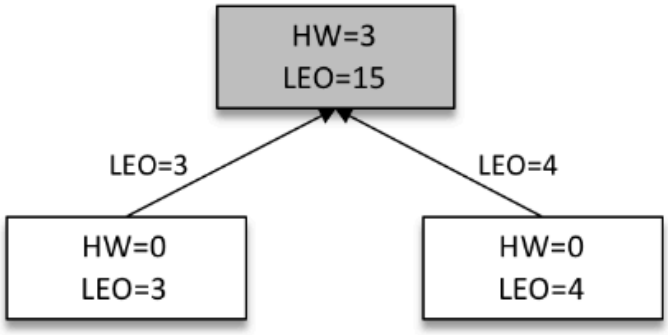


情形2

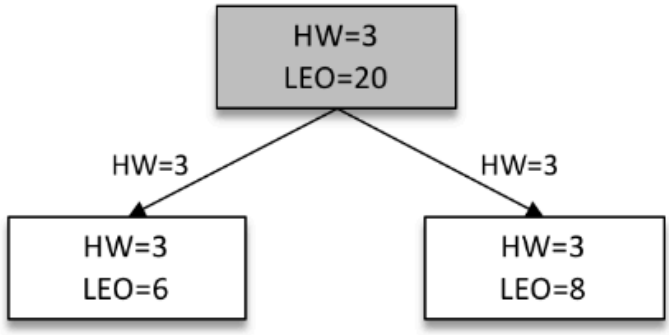
之后 follower 副本（不带阴影的方框）向 leader 副本拉取消息，在拉取的请求中会带有自身的 LEO 信息，这个 LEO 信息对应的是 FetchRequest 请求中的 fetch_offset。leader 副本返回给 follower 副本相应的消息，并且还带有自身的 HW 信息，如上图（右）所示，这个 HW 信息对应的是 FetchResponse 中的 high_watermark。

此时两个 follower 副本各自拉取到了消息，并更新各自的 LEO 为3和4。与此同时，**follower 副本还会更新自己的 HW，更新 HW 的算法是比较当前 LEO 和 leader 副本中传送过来的HW的值，取较小值作为自己的 HW 值。**当前两个 follower 副本的 HW 都等于0（min(0,0) = 0）。

接下来 follower 副本再次请求拉取 leader 副本中的消息，如下图（左）所示。



情形3



情形4

此时 leader 副本收到来自 follower 副本的 FetchRequest 请求，其中带有 LEO 的相关信息，选取其中的最小值作为新的 HW，即 min(15,3,4)=3。然后连同消息和 HW 一起返回 FetchResponse 给 follower 副本，如上图（右）所示。注意 leader 副本的 HW 是一个很重要的东西，因为它直接影响了分区数据对消费者的可见性。

两个 follower 副本在收到新的消息之后更新 LEO 并且更新自己的 HW 为3（min(LEO,3)=3）。

4、Kafka是如何保障数据不丢失的？（理解背诵）

该问题已经成为了Kafka面试的惯例，如同Java的**HashMap**，属于高频出现的面试问题。那么，我们该怎么理解这个问题呢？问题是**Kafka如何保障数据不丢失**，即**Kafka的Broker提供了什么机制保证数据不丢失的。**

其实对于Kafka的Broker而言，Kafka 的**复制机制**和**分区的多副本**架构是Kafka 可靠性保证的核心。把消息写入多个副本可以使Kafka 在发生崩溃时仍能保证消息的持久性。

搞清楚了问题的核心，再来看一下该怎么回答这个问题：主要包括三个方面

> **1.Topic 副本因子个数：replication.factor >= 3**
>
> **2.同步副本列表(ISR)：min.insync.replicas = 2**
>
> **3.禁用unclean选举： unclean.leader.election.enable=false**

下面将会逐步分析上面的三个配置：

- **副本因子**

Kafka的topic是可以分区的，并且可以为分区配置多个副本，该配置可以通过`replication.factor`参数实现。Kafka中的分区副本包括两种类型：领导者副本（Leader Replica）和追随者副本（Follower Replica），每个分区在创建时都要选举一个副本作为领导者副本，其余的副本自动变为追随者副本。在 Kafka 中，追随者副本是不对外提供服务的，也就是说，任何一个追随者副本都不能响应消费者和生产者的读写请求。所有的请求都必须由领导者副本来处理。换句话说，所有的读写请求都必须发往领导者副本所在的 Broker，由该 Broker 负责处理。追随者副本不处理客户端请求，它唯一的任务就是从领导者副本**异步拉取**消息，并写入到自己的提交日志中，从而实现与领导者副本的同步。

一般来说，副本设为3可以满足大部分的使用场景，也有可能是5个副本(比如银行)。如果副本因子为N，那么在N-1个broker 失效的情况下，仍然能够从主题读取数据或向主题写入数据。所以，更高的副本因子会带来更高的可用性、可靠性和更少的故障。另一方面，副本因子N需要至少N个broker ，而且会有N个数据副本，也就是说它们会占用N倍的磁盘空间。实际生产环境中一般会在可用性和存储硬件之间作出权衡。

除此之外，副本的分布同样也会影响可用性。默认情况下，Kafka会确保分区的每个副本分布在不同的Broker上，但是如果这些Broker在同一个机架上，一旦机架的交换机发生故障，分区就会不可用。所以建议把Broker分布在不同的机架上，可以使用**broker.rack**参数配置Broker所在机架的名称。

- **同步副本列表**

In-sync replica(ISR)称之为同步副本，ISR中的副本都是与Leader进行同步的副本，所以不在该列表的follower会被认为与Leader是不同步的。那么，ISR中存在是什么副本呢？首先可以明确的是：Leader副本总是存在于ISR中。而follower副本是否在ISR中，取决于该follower副本是否与Leader副本保持了“同步”。

Kafka的broker端有一个参数`replica.lag.time.max.ms`, 该参数表示follower副本滞后与Leader副本的最长时间间隔，默认是10秒。这就意味着，只要follower副本落后于leader副本的时间间隔不超过10秒，就可以认为该follower副本与leader副本是同步的，所以哪怕当前follower副本落后于Leader副本几条消息，只要在10秒之内赶上Leader副本，就不会被踢出出局。

可以看出ISR是一个动态的，所以即便是为分区配置了3个副本，还是会出现同步副本列表中只有一个副本的情况(其他副本由于不能够与leader及时保持同步，被移出ISR列表)。如果这个同步副本变为不可用，我们必须在**可用性**和**一致性**之间作出选择(CAP理论)。

根据Kafka 对可靠性保证的定义，消息只有在被写入到所有同步副本之后才被认为是已提交的。但如果这里的“所有副本”只包含一个同步副本，那么在这个副本变为不可用时，数据就会丢失。如果要确保已提交的数据被写入不止一个副本，就需要把最小同步副本数量设置为大一点的值。对于一个包含3 个副本的主题分区，如果**min.insync.replicas=2** ，那么至少要存在两个同步副本才能向分区写入数据。

如果进行了上面的配置，此时必须要保证ISR中至少存在两个副本，如果ISR中的副本个数小于2，那么Broker就会停止接受生产者的请求。尝试发送数据的生产者会收到**NotEnoughReplicasException**异常，消费者仍然可以继续读取已有的数据。

- **禁用unclean选举**

选择一个同步副本列表中的分区作为leader 分区的过程称为**clean leader election**。注意，这里要与在非同步副本中选一个分区作为leader分区的过程区分开，在非同步副本中选一个分区作为leader的过程称之为**unclean leader election**。由于ISR是动态调整的，所以会存在ISR列表为空的情况，通常来说，非同步副本落后 Leader 太多，因此，如果选择这些副本作为新 Leader，就可能出现数据的丢失。毕竟，这些副本中保存的消息远远落后于老 Leader 中的消息。在 Kafka 中，选举这种副本的过程可以通过Broker 端参数 **unclean.leader.election.enable **控制是否允许 Unclean 领导者选举。开启 Unclean 领导者选举可能会造成数据丢失，但好处是，它使得分区 Leader 副本一直存在，不至于停止对外提供服务，因此提升了高可用性。反之，禁止 Unclean Leader 选举的好处在于维护了数据的一致性，避免了消息丢失，但牺牲了高可用性。分布式系统的CAP理论说的就是这种情况。

不幸的是，**unclean leader election**的选举过程仍可能会造成数据的不一致，因为同步副本并不是**完全**同步的。由于复制是**异步**完成的，因此无法保证follower可以获取最新消息。比如Leader分区的最后一条消息的offset是100，此时副本的offset可能不是100，这受到两个参数的影响：

> - **replica.lag.time.max.ms**：同步副本滞后与leader副本的时间
> - **zookeeper.session.timeout.ms**：与zookeeper会话超时时间

简而言之，如果我们允许不同步的副本成为leader，那么就要承担丢失数据和出现数据不一致的风险。如果不允许它们成为leader，那么就要接受较低的可用性，因为我们必须等待原先的首领恢复到可用状态。

关于unclean选举，不同的场景有不同的配置方式。对**数据质量和数据一致性**要求较高的系统会禁用这种unclean的leader选举(比如银行)。如果在**可用性**要求较高的系统里，比如实时点击流分析系统， 一般不会禁用unclean的leader选举。

5、如何解决Kafka数据丢失问题？（理解背诵）

你可能会问：这个问题跟上一个问题有什么区别呢？其实一般在面试问题中可以理解成一个问题。之所以在这里做出区分，是因为两者的解决方式不一样。**上一个问题是从Kafka的Broker侧来看待数据丢失的问题，而当前的问题是从Kafka的生产者与消费者的角度来看待数据丢失的问**

题。

先来看一下如何回答这个问题：主要包括两个方面：

> **Producer**
>
> - retries=Long.MAX_VALUE
>
> 设置 retries 为一个较大的值。这里的 retries 同样是 Producer 的参数，对应前面提到的 Producer 自动重试。当出现网络的瞬时抖动时，消息发送可能会失败，此时配置了 retries > 0 的 Producer 能够自动重试消息发送，避免消息丢失。
> - acks=all
>
> 设置 acks = all。acks 是 Producer 的一个参数，代表了你对“已提交”消息的定义。如果设置成 all，则表明所有副本 Broker 都要接收到消息，该消息才算是“已提交”。这是最高等级的“已提交”定义。
> - max.in.flight.requests.per.connections=1
>
> 该参数指定了生产者在收到服务器响应之前可以发送多少个消息。它的值越高，就会占用越多的内存，不过也会提升吞吐量。把它设为1 可以保证消息是按照发送的顺序写入服务器的，即使发生了重试。
> - Producer要使用带有回调通知的API，也就是说不要使用 producer.send(msg)，而要使用 producer.send(msg, callback)。
> - 其他错误处理
>
> 使用生产者内置的重试机制，可以在不造成消息丢失的情况下轻松地处理大部分错误，不过
> 仍然需要处理其他类型的错误，例如消息大小错误、序列化错误等等。
>
> **Consumer**
>
> - 禁用自动提交：enable.auto.commit=false
> - 消费者处理完消息之后再提交offset
> - 配置auto.offset.reset
>
> 这个参数指定了在没有偏移量可提交时(比如消费者第I次启动时)或者请求的偏移量在broker上不存在时(比如数据被删了)，消费者会做些什么。
>
> 这个参数有两种配置。一种是**earliest**：消费者会从分区的开始位置读取数据，不管偏移量是否有效，这样会导致消费者读取大量的重复数据，但可以保证最少的数据丢失。一种是**latest(默认)**，如果选择了这种配置， 消费者会从分区的末尾开始读取数据，这样可以减少重复处理消息，但很有可能会错过一些消息。

6、Kafka可以保障永久不丢失数据吗？（理解）

上面分析了一些保障数据不丢失的措施，在一定程度上可以避免数据的丢失。但是请注意：**Kafka 只对“已提交”的消息（committed message）做有限度的持久化保证**。所以说，Kafka不能够完全保证数据不丢失，需要做出一些权衡。

首先，要理解什么是**已提交的消息**，当 Kafka 的若干个 Broker 成功地接收到一条消息并写入到日志文件后，它们会告诉生产者程序这条消息已成功提交。此时，这条消息在 Kafka 看来就正式变为**已提交**消息了。所以说无论是ack=all，还是ack=1,不论哪种情况，Kafka 只对已提交的消息做持久化保证这件事情是不变的。

其次，要理解**有限度的持久化保证**，也就是说 Kafka 不可能保证在任何情况下都做到不丢失消息。必须保证Kafka的Broker是可用的，换句话说，假如消息保存在 N 个 Kafka Broker 上，那么这个前提条件就是这 N 个 Broker 中至少有 1 个存活。只要这个条件成立，Kafka 就能保证你的这条消息永远不会丢失。

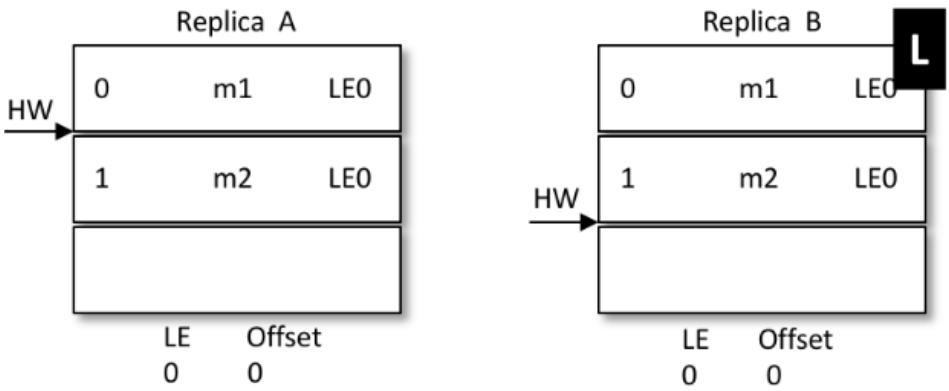
总结一下，Kafka 是能做到不丢失消息的，只不过这些消息必须是已提交的消息，而且还要满足一定的条件。

7、Kafka在可靠性方面做了哪些改进？（理解）

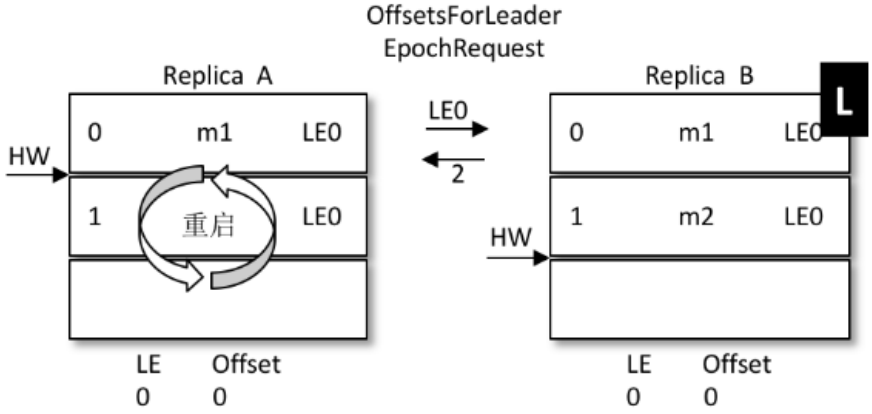
HW 是 High Watermark 的缩写，俗称高水位，它标识了一个特定的消息偏移量（offset），消费者只能拉取到这个 offset 之前的消息。分区 ISR 集合中的每个副本都会维护自身的 LEO，而 ISR 集合中最小的 LEO 即为分区的 HW，对消费者而言只能消费 HW 之前的消息。

leader epoch 代表 leader 的纪元信息（epoch），初始值为0。每当 leader 变更一次，leader epoch 的值就会加1，相当于为 leader 增设了一个版本号。每个副本中还会增设一个矢量 <LeaderEpoch => StartOffset>，其中 StartOffset 表示当前 LeaderEpoch 下写入的第一条消息的偏移量。

假设有两个节点A和B，B是leader节点，里面的数据如图：

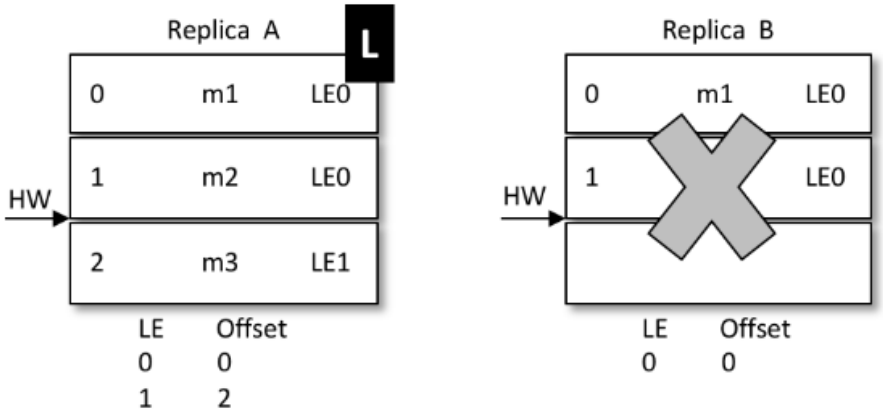


A发生重启，之后A不是先忙着截断日志而是先发送OffsetsForLeaderEpochRequest请求给B，B作为目前的leader在收到请求之后会返回当前的LEO（LogEndOffset，注意图中LE0和LEO的不同），与请求对应的响应为OffsetsForLeaderEpochResponse。如果 A 中的 LeaderEpoch（假设为 LE_A）和 B 中的不相同，那么 B 此时会查找 LeaderEpoch 为 LE_A+1 对应的 StartOffset 并返回给 A

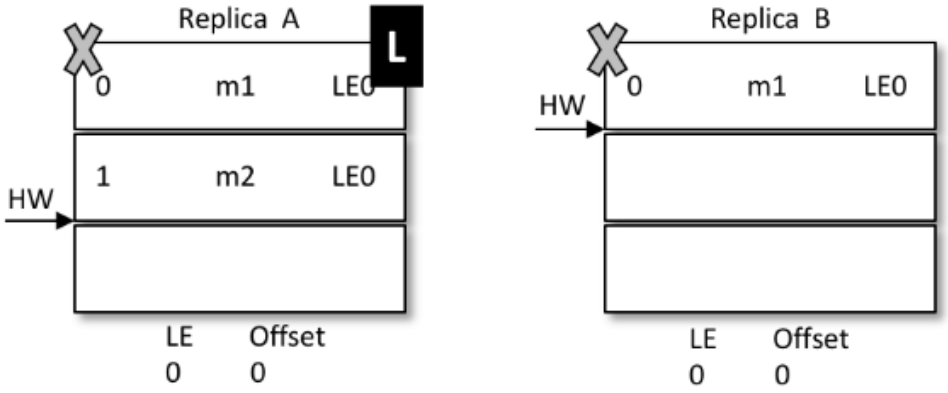


如上图所示，A 在收到2之后发现和目前的 LEO 相同，也就不需要截断日志了，以此来保护数据的完整性。

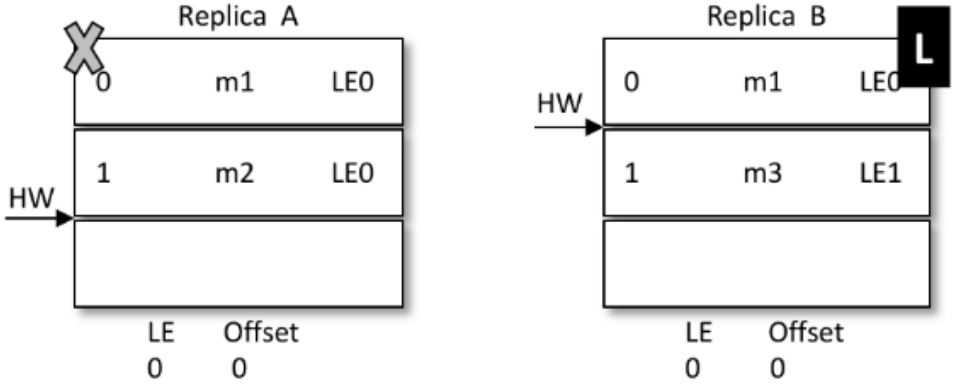
再如，之后 B 发生了宕机，A 成为新的 leader，那么对应的 LE=0 也变成了 LE=1，对应的消息 m2 此时就得到了保留。后续的消息都可以以 LE1 为 LeaderEpoch 陆续追加到 A 中。这个时候A就会有两个LE，第二LE所记录的Offset从2开始。如果B恢复了，那么就会从A中获取到LE+1的Offset为2的值返回给B。



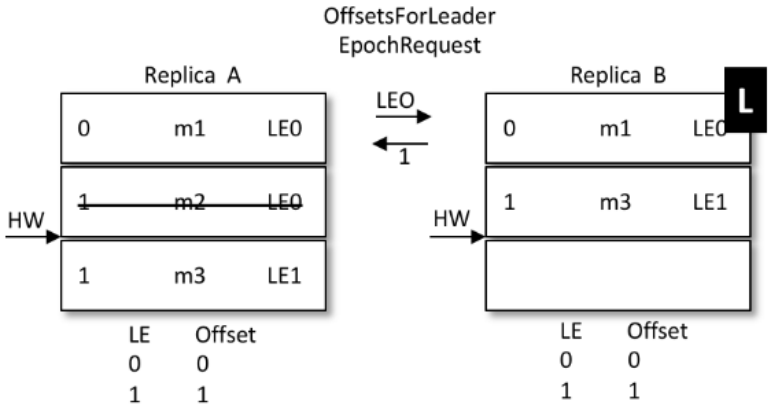
再来看看LE如何解决数据不一致的问题：
当前 A 为 leader，B 为 follower，A 中有2条消息 m1 和 m2，而 B 中有1条消息 m1。假设 A 和 B 同时“挂掉”，然后 B 第一个恢复过来并成为新的 leader。



之后 B 写入消息 m3，并将 LEO 和 HW 更新至2，如下图所示。注意此时的 LeaderEpoch 已经从 LE0 增至 LE1 了。



紧接着 A 也恢复过来成为 follower 并向 B 发送 OffsetsForLeaderEpochRequest 请求，此时 A 的 LeaderEpoch 为 LE0。B 根据 LE0 查询到对应的 offset 为1并返回给 A，A 就截断日志并删除了消息 m2，如下图所示。之后 A 发送 FetchRequest 至 B 请求来同步数据，最终A和B中都有两条消息 m1 和 m3，HW 和 LEO都为2，并且 LeaderEpoch 都为 LE1，如此便解决了数据不一致的问题。



8、如何保障Kafka中的消息是有序的？（理解背诵）

首先需要明确的是：**Kafka的主题是分区有序的**，如果一个主题有多个分区，那么Kafka会按照key将其发送到对应的分区中，所以，对于给定的key，与其对应的record在分区内是有序的。

Kafka可以保证同一个分区里的消息是有序的，即生产者按照一定的顺序发送消息，Broker就会按照这个顺序将他们写入对应的分区中，同理，消费者也会按照这个顺序来消费他们。

在一些场景下，消息的顺序是非常重要的。比如，**先存钱再取钱**与**先取钱再存钱**是截然不同的两种结果。

上面的问题中提到一个参数**max.in.flight.requests.per.connections=1**,该参数的作用是在重试次数大于等于1时，保证数据写入的顺序。如果该参数不为1，那么当第一个批次写入失败时，第二个批次写入成功，Broker会重试写入第一个批次，如果此时第一个批次重试写入成功，那么这两个批次消息的顺序就反过来了。

一般来说，如果对消息的顺序有要求，那么在为了保障数据不丢失，需要先设置发送重试次数retries>0，同时需要把max.in.flight.requests.per.connections参数设为1，这样在生产者尝试发送第一批消息时，就不会有其他的消息发送给broker，虽然会影响吞吐量，但是可以保证消息的顺序。

除此之外，还可以使用单分区的Topic，但是会严重影响吞吐量。

9、如何确定合适的Kafka主题的分区数量？（理解）

选择合适的分区数量可以达到高度并行读写和负载均衡的目的，在分区上达到均衡负载是实现吞吐量的关键。需要根据每个分区的生产者和消费者的期望吞吐量进行估计。

举个栗子：假设期望读取数据的速率(吞吐量)为**1GB/Sec**，而一个消费者的读取速率为**50MB/Sec**，此时至少需要20个分区以及20个消费者(一个消费者组)。同理，如果期望生产数据的速率为**1GB/Sec**，而每个生产者的生产速率为**100MB/Sec**，此时就需要有10个分区。在这种情况下，如果设置20个分区，既可以保障**1GB/Sec**的生产速率，也可以保障消费者的吞吐量。通常需要将分区的数量调整为消费者或者生产者的数量，只有这样才可以同时实现生产者和消费者的吞吐量。

一个简单的计算公式为：**分区数 = max(生产者数量，消费者数量)**

- 生产者数量=整体生产吞吐量/每个生产者对单个分区的最大生产吞吐量
- 消费者数量=整体消费吞吐量/每个消费者从单个分区消费的最大吞吐量

10、为什么Kafka不支持读写分离？（了解）

因为这样有两个明显的缺点：

1. **数据一致性问题。**数据从主节点转到从节点必然会有一个延时的时间窗口，这个时间窗口会导致主从节点之间的数据不一致。
2. **延时问题。**数据从写入主节点到同步至从节点中的过程需要经历网络→主节点内存→主节点磁盘→网络→从节点内存→从节点磁盘这几个阶段。对延时敏感的应用而言，主写从读的功能并不太适用。

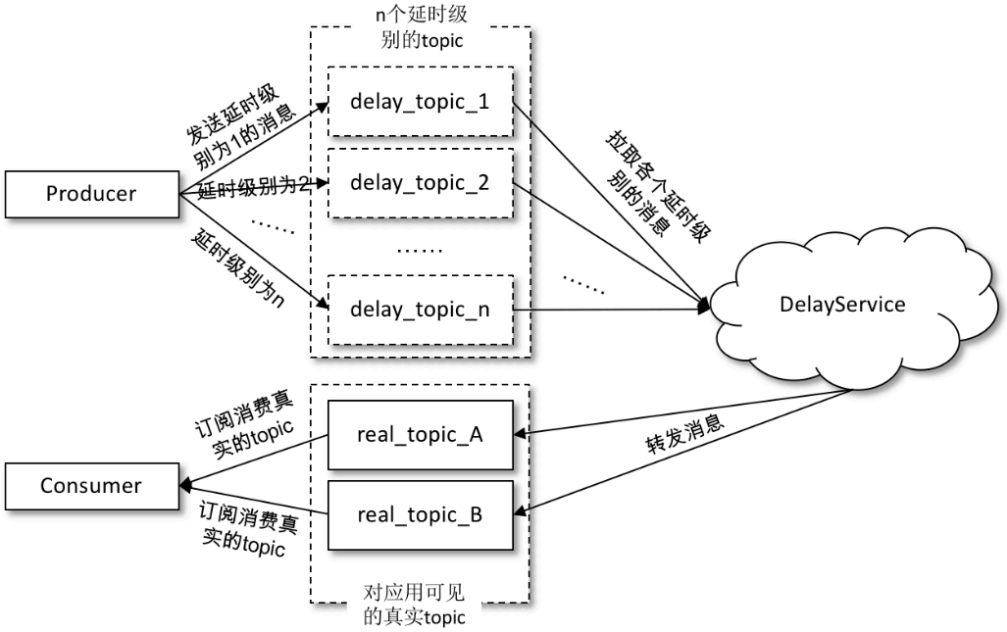
对于Kafka来说，必要性不是很高，**因为在Kafka集群中，如果存在多个副本，经过合理的配置，可以让leader副本均匀的分布在各个broker上面，使每个 broker 上的读写负载都是一样的。**

11、Kafka中的延迟队列怎么实现（理解）

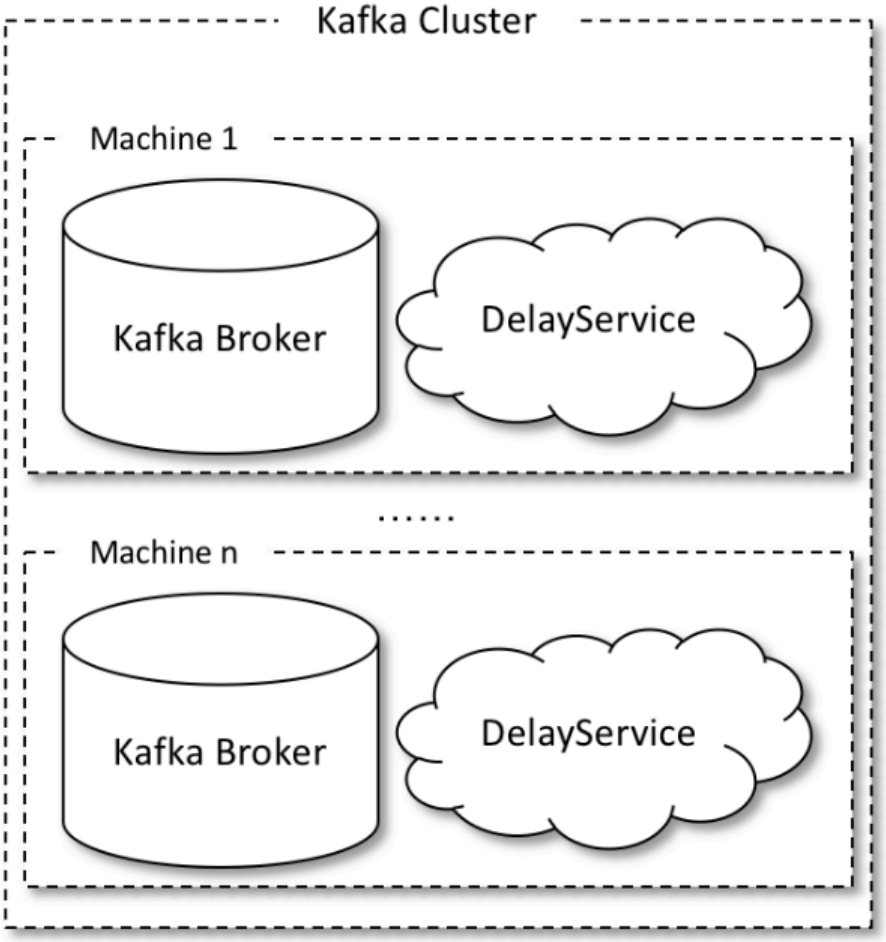
在发送延时消息的时候并不是先投递到要发送的真实主题（real_topic）中，而是先投递到一些 Kafka 内部的主题（delay_topic）中，这些内部主题对用户不可见，然后通过一个自定义的服务拉取这些内部主题中的消息，并将满足条件的消息再投递到要发送的真实的主题中，消费者所订阅的还是真实的主题。

如果采用这种方案，那么一般是按照不同的延时等级来划分的，比如设定5s、10s、30s、1min、2min、5min、10min、20min、30min、45min、1hour、2hour这些按延时时间递增的延时等级，延时的消息按照延时时间投递到不同等级的主题中，投递到同一主题中的消息的延时时间会被强转为与此主题延时等级一致的延时时间，这样延时误差控制在两个延时等级的时间差范围之内（比如延时时间为17s的消息投递到30s的

延时主题中，之后按照延时时间为30s进行计算，延时误差为13s）。虽然有一定的延时误差，但是误差可控，并且这样只需增加少许的主题就能实现延时队列的功能。

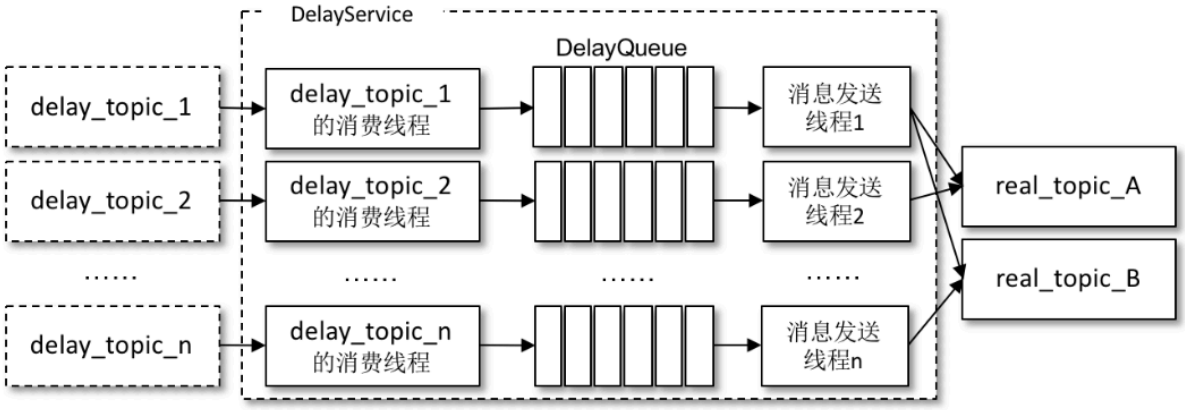


发送到内部主题（delay_topic_*）中的消息会被一个独立的 DelayService 进程消费，这个 DelayService 进程和 Kafka broker 进程以一对一的配比进行同机部署（参考下图），以保证服务的可用性。



针对不同延时级别的主题，在 DelayService 的内部都会有单独的线程来进行消息的拉取，以及单独的 DelayQueue（这里用的是 JUC 中 DelayQueue）进行消息的暂存。与此同时，在 DelayService 内部还会有专门的消息发送线程来获取 DelayQueue 的消息并转发到真实的主题

中。从消费、暂存再到转发，线程之间都是一一对应的关系。如下图所示，DelayService 的设计应当尽量保持简单，避免锁机制产生的隐患。



为了保障内部 DelayQueue 不会因为未处理的消息过多而导致内存的占用过大，DelayService 会对主题中的每个分区进行计数，当达到一定的阈值之后，就会暂停拉取该分区中的消息。

因为一个主题中一般不止一个分区，分区之间的消息并不会按照投递时间进行排序，DelayQueue的作用是将消息按照再次投递时间进行有序排序，这样下游的消息发送线程就能够按照先后顺序获取最先满足投递条件的消息。

12、Kafka中怎么实现死信队列和重试队列？（理解）

死信可以看作消费者不能处理收到的消息，也可以看作消费者不想处理收到的消息，还可以看作不符合处理要求的消息。比如消息内包含的消息内容无法被消费者解析，为了确保消息的可靠性而不被随意丢弃，故将其投递到死信队列中，这里的死信就可以看作消费者不能处理的消息。再比如超过既定的重试次数之后将消息投入死信队列，这里就可以将死信看作不符合处理要求的消息。

重试队列其实可以看作一种回退队列，具体指消费端消费消息失败时，为了防止消息无故丢失而重新将消息回滚到 broker 中。与回退队列不同的是，重试队列一般分成多个重试等级，每个重试等级一般也会设置重新投递延时，重试次数越多投递延时就越大。

理解了他们的概念之后我们就可以为每个主题设置重试队列，消息第一次消费失败入重试队列 Q1，Q1 的重新投递延时为5s，5s过后重新投递该消息；如果消息再次消费失败则入重试队列 Q2，Q2 的重新投递延时为10s，10s过后再次投递该消息。

然后再设置一个主题作为死信队列，重试越多次重新投递的时间就越久，并且需要设置一个上限，超过投递次数就进入死信队列。重试队列与延时队列有相同的地方，都需要设置延时级别。

13、Kafka中怎么做消息审计？（了解）

消息审计是指在消息生产、存储和消费的整个过程之间对消息个数及延迟的审计，以此来检测是否有数据丢失、是否有数据重复、端到端的延迟又是多少等内容。

目前与消息审计有关的产品也有多个，比如 Chaperone（Uber）、Confluent Control Center、Kafka Monitor（LinkedIn），它们主要通过在消息体（value 字段）或在消息头（headers 字段）中内嵌消息对应的时间戳 timestamp 或全局的唯一标识 ID（或者是两者兼备）来实现消息的审计功能。

内嵌 timestamp 的方式主要是设置一个审计的时间间隔 time_bucket_interval（可以自定义设置几秒或几分钟），根据这个 time_bucket_interval 和消息所属的 timestamp 来计算相应的时间桶（time_bucket）。

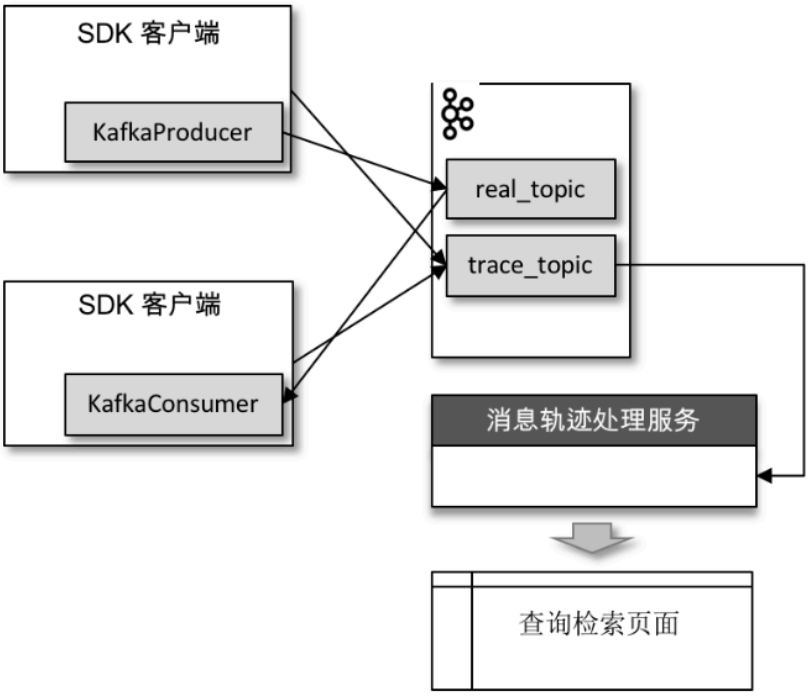
内嵌 ID 的方式就更加容易理解了，对于每一条消息都会被分配一个全局唯一标识 ID。如果主题和相应的分区固定，则可以为每个分区设置一个全局的 ID。当有消息发送时，首先获取对应的 ID，然后内嵌到消息中，最后才将它发送到 broker 中。消费者进行消费审计时，可以判断出哪条消息丢失、哪条消息重复。

14、Kafka中怎么做消息轨迹？（了解）

消息轨迹指的是一条消息从生产者发出，经由 broker 存储，再到消费者消费的整个过程中，各个相关节点的状态、时间、地点等数据汇聚而成的完整链路信息。生产者、broker、消费者这3个角色在处理消息的过程中都会在链路中增加相应的信息，将这些信息汇聚、处理之后就可以查询任意消息的状态，进而为生产环境中的故障排除提供强有力的数据支持。

对消息轨迹而言，最常见的实现方式是封装客户端，在保证正常生产消费的同时添加相应的轨迹信息埋点逻辑。无论生产，还是消费，在执行之后都会有相应的轨迹信息，我们需要将这些信息保存起来。

我们同样可以将轨迹信息保存到 Kafka 的某个主题中，比如下图中的主题 trace_topic。



生产者在将消息正常发送到用户主题 real_topic 之后（或者消费者在拉取到消息消费之后）会将轨迹信息发送到主题 trace_topic 中。

15、怎么计算Lag？(注意read_uncommitted和read_committed状态下的不同)（理解）

如果消费者客户端的 isolation.level 参数配置为“read_uncommitted”(默认）,它对应的 Lag 等于HW – ConsumerOffset 的值，其中 ConsumerOffset 表示当前的消费位移。

如果这个参数配置为“read_committed”，那么就要引入 LSO 来进行计算了。LSO 是 LastStableOffset 的缩写,它对应的 Lag 等于 LSO – ConsumerOffset 的值。

- 首先通过 DescribeGroupsRequest 请求获取当前消费组的元数据信息，当然在这之前还会通过 FindCoordinatorRequest 请求查找消费组对应的 GroupCoordinator。
- 接着通过 OffsetFetchRequest 请求获取消费位移 ConsumerOffset。
- 然后通过 KafkaConsumer 的 endOffsets(Collection partitions)方法（对应于 ListOffsetRequest 请求）获取 HW（LSO）的值。
- 最后通过 HW 与 ConsumerOffset 相减得到分区的 Lag，要获得主题的总体 Lag 只需对旗下的各个分区累加即可。

16、Kafka有哪些指标需要着重关注？（了解）

比较重要的 Broker 端 JMX 指标：

- BytesIn/BytesOut：即 Broker 端每秒入站和出站字节数。你要确保这组值不要接近你的网络带宽，否则这通常都表示网卡已被“打满”，很容易出现网络丢包的情形。
- NetworkProcessorAvgIdlePercent：即网络线程池线程平均的空闲比例。通常来说，你应该确保这个 JMX 值长期大于 30%。如果小于这个值，就表明你的网络线程池非常繁忙，你需要通过增加网络线程数或将负载转移给其他服务器的方式，来给该 Broker 减负。
- RequestHandlerAvgIdlePercent：即 I/O 线程池线程平均的空闲比例。同样地，如果该值长期小于 30%，你需要调整 I/O 线程池的数量，或者减少 Broker 端的负载。
- UnderReplicatedPartitions：即未充分备份的分区数。所谓未充分备份，是指并非所有的 Follower 副本都和 Leader 副本保持同步。一旦出现了这种情况，通常都表明该分区有可能会出现数据丢失。因此，这是一个非常重要的 JMX 指标。
- ISRShrink/ISRExpand：即 ISR 收缩和扩容的频次指标。如果你的环境中出现 ISR 中副本频繁进出的情形，那么这组值一定是很高的。这时，你要诊断下副本频繁进出 ISR 的原因，并采取适当的措施。
- ActiveControllerCount：即当前处于激活状态的控制器的数量。正常情况下，Controller 所在 Broker 上的这个 JMX 指标值应该是 1，其他 Broker 上的这个值是 0。如果你发现存在多台 Broker 上该值都是 1 的情况，一定要赶快处理，处理方式主要是查看网络连通性。这种情况通常表明集群出现了脑裂。脑裂问题是非常严重的分布式故障，Kafka 目前依托 ZooKeeper 来防止脑裂。但一旦出现脑裂，Kafka 是无法保证正常工作的。

17、Kafka如何实现高吞吐的原理？（理解背诵）

1. **分区**
   kafka是个分布式集群的系统，整个系统可以包含多个broker，也就是多个服务器实例。每个主题topic会有多个分区，kafka将分区均匀地分配到整个集群中，当生产者向对应主题传递消息，消息通过负载均衡机制传递到不同的分区以减轻单个服务器实例的压力。一个Consumer Group中可以有多个consumer，多个consumer可以同时消费不同分区的消息，大大的提高了消费者的并行消费能力。但是一个分区中的消息只能被一个Consumer Group中的一个consumer消费。
2. **网络传输上减少开销**
   批量发送：在发送消息的时候，kafka不会直接将少量数据发送出去，否则每次发送少量的数据会增加网络传输频率，降低网络传输效率。kafka会先将消息缓存在内存中，当超过一个的大小或者超过一定的时间，那么会将这些消息进行批量发送。
   端到端压缩：当然网络传输时数据量小也可以减小网络负载，kafaka会将这些批量的数据进行压缩，将一批消息打包后进行压缩，发送

broker服务器后，最终这些数据还是提供给消费者用，所以数据在服务器上还是保持压缩状态，不会进行解压，而且频繁的压缩和解压也会降低性能，最终还是以压缩的方式传递到消费者的手上。

3. **顺序读写**

kafka 的消息是不断追加到文件中的，这个特性使 kafka 可以充分利用磁盘的顺序读写性能，顺序读写不需要硬盘磁头的寻道时间，只需很少的扇区旋转时间，所以速度远快于随机读写

Kafka 官方给出了测试数据(Raid-5，7200rpm)：

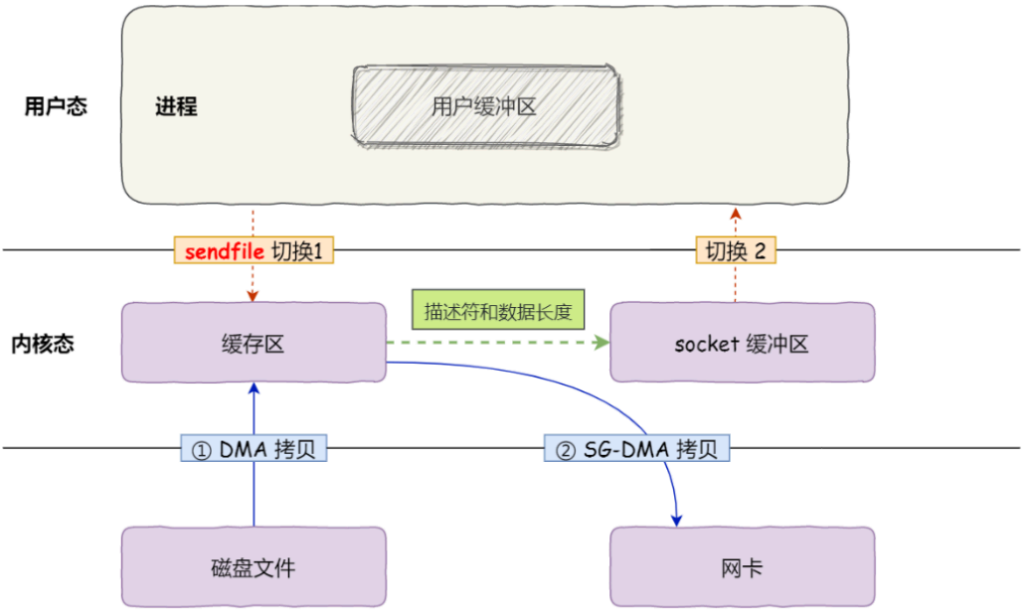
顺序 I/O：600MB/s

随机 I/O：100KB/s

4. **零拷贝技术**

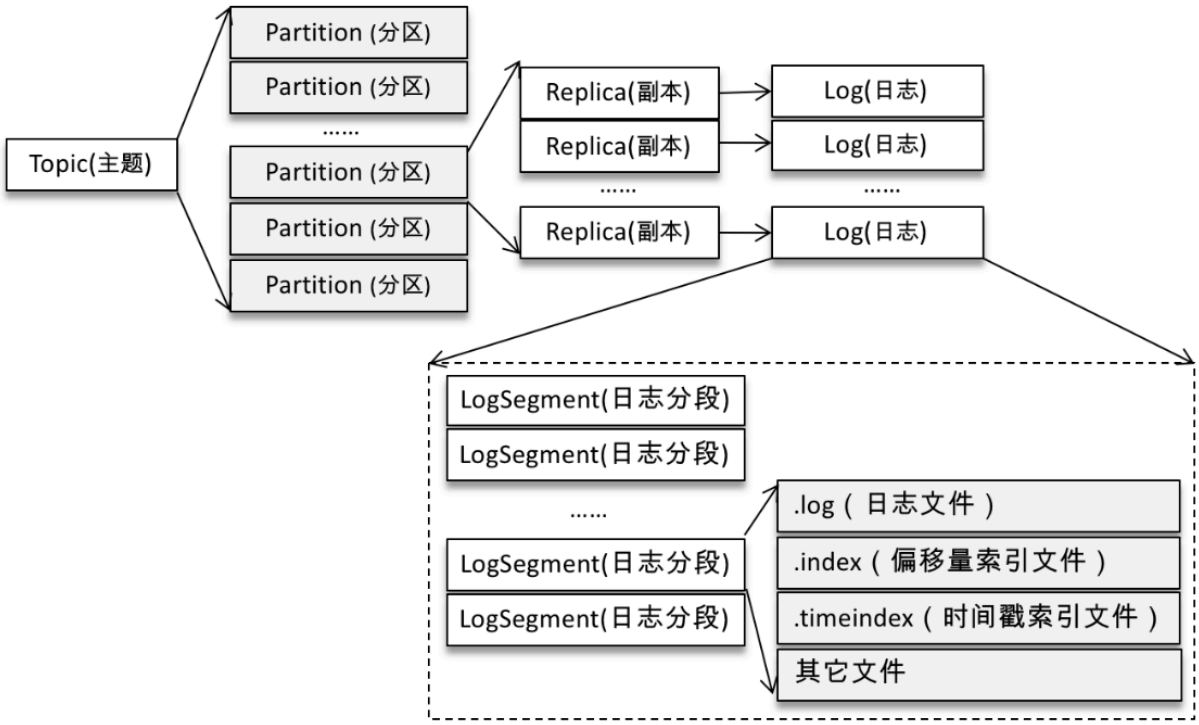
零拷贝是指将数据直接从磁盘复制到网卡设备中，而不需要经由CPU之手。大大提高了应用程序的性能，减少了内核和用户模式之间的上下文切换。数据只经过了2次copy就从磁盘传送出去了。

零拷贝将文件内容从磁盘通过DMA引擎复制到内核缓冲区，而且没有把数据复制到socket缓冲区，只是将数据位置和长度信息的描述符复制到了socket缓存区，然后直接将数据传输到网络接口，最后发送。这样大大减小了拷贝的次数，提高了效率。kafka正是调用linux系统给出的sendfile系统调用来使用零拷贝。Java中的系统调用给出的是FileChannel.transferTo接口。



5. 优秀的文件存储机制

如果分区规则设置得合理，那么所有的消息可以均匀地分布到不同的分区中，这样就可以实现水平扩展。不考虑多副本的情况，一个分区对应一个日志（Log）。为了防止 Log 过大，Kafka 又引入了日志分段（LogSegment）的概念，将 Log 切分为多个 LogSegment，相当于一个巨型文件被平均分配为多个相对较小的文件，这样也便于消息的维护和清理。

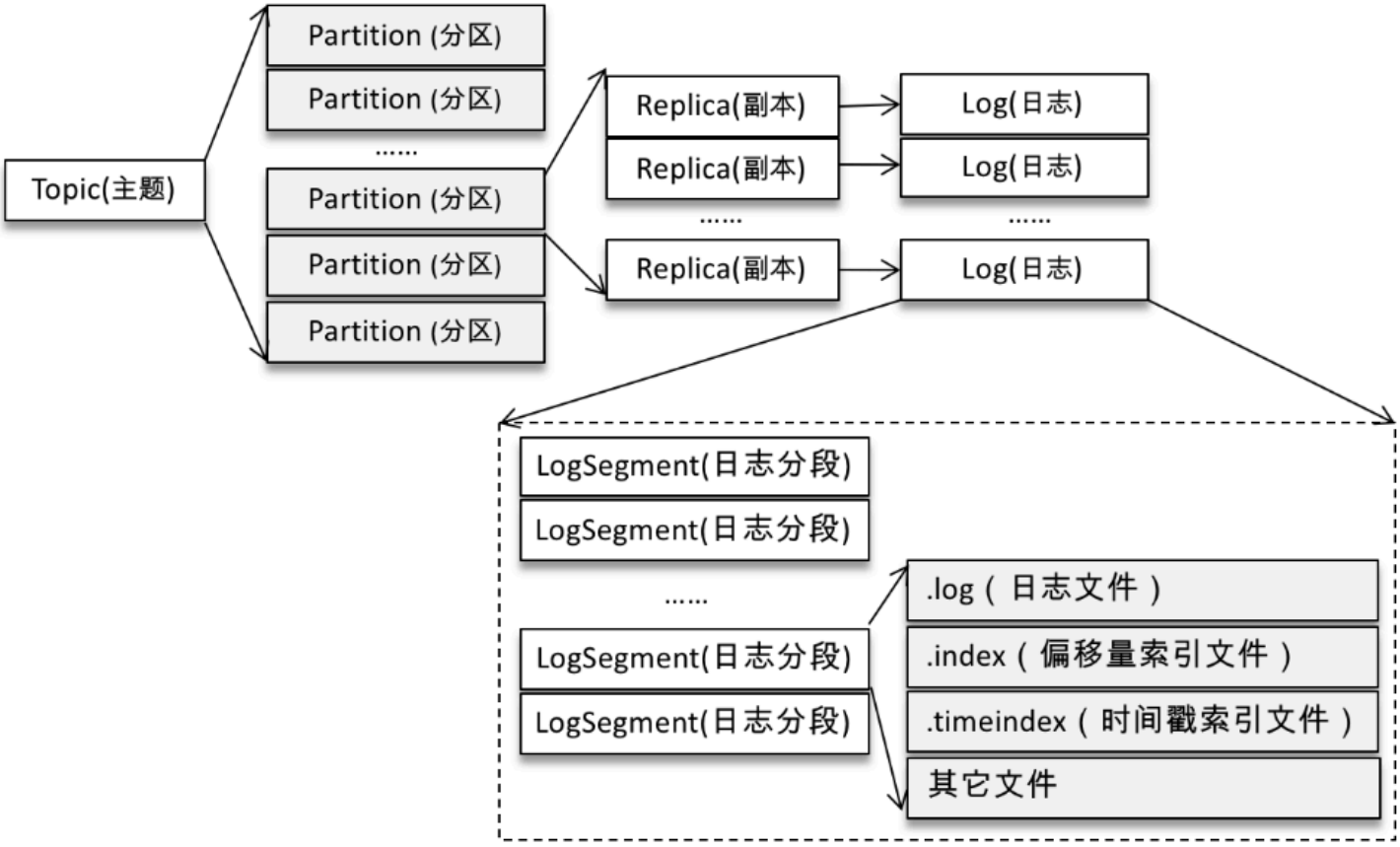


Kafka 中的索引文件以稀疏索引（sparse index）的方式构造消息的索引，它并不保证每个消息在索引文件中都有对应的索引项。每当写入一定量（由 broker 端参数 log.index.interval.bytes 指定，默认值为4096，即 4KB）的消息时，偏移量索引文件和时间戳索引文件分别增加一个偏移

量索引项和时间戳索引项，增大或减小 log.index.interval.bytes 的值，对应地可以增加或缩小索引项的密度。

18、Kafka里面存的数据格式是什么样的？（了解）

前面说过kafka的日志目录结构，这里我们再回顾一下



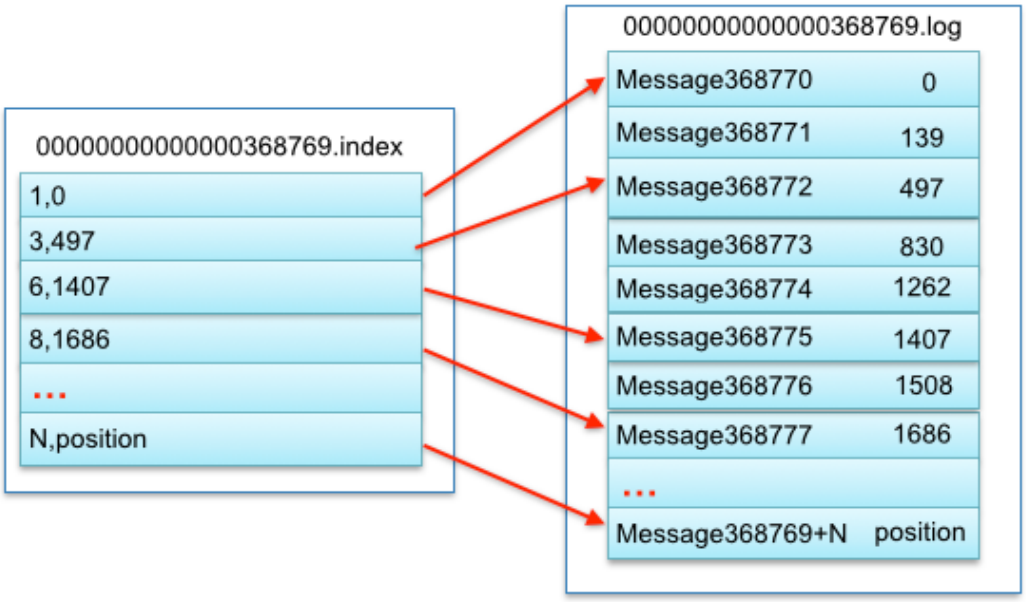
因为我们知道真实的数据都是存储再Segment段中的，所以我们这里重点剖析一下Segment的内容。**segment file由2部分组成，分别为index file和data file，这两个文件是一一对应的，后**

缀".index"和".log"分别表示索引文件和数据文件；

segment file命名规则：partition的第一个segment从0开始，后续每个segment文件名为上一个segment文件最后一条消息的offset,ofsset的数值最大为64位（long类型），20位数字字符长度，没有数字用0填充。如下图所示：

```
[root@datanode33 fast-query-userflow-2]# ll
total 9656168
-rw-r--r-- 1 root root    292320 Oct 27 15:13 00000000000000000000.index
-rw-r--r-- 1 root root 536879579 Oct 27 15:13 00000000000000000000.log
-rw-r--r-- 1 root root    293352 Oct 27 15:22 00000000000000935097.index
-rw-r--r-- 1 root root 536875566 Oct 27 15:22 00000000000000935097.log
-rw-r--r-- 1 root root    292592 Oct 27 16:09 00000000000001873473.index
-rw-r--r-- 1 root root 536880135 Oct 27 16:09 00000000000001873473.log
-rw-r--r-- 1 root root    293216 Oct 27 16:15 00000000000002809932.index
-rw-r--r-- 1 root root 536881403 Oct 27 16:15 00000000000002809932.log
-rw-r--r-- 1 root root    293360 Oct 27 16:39 00000000000003748645.index
-rw-r--r-- 1 root root 536876611 Oct 27 16:39 00000000000003748645.log
-rw-r--r-- 1 root root    292896 Oct 27 16:47 00000000000004687014.index
-rw-r--r-- 1 root root 536875803 Oct 27 16:47 00000000000004687014.log
-rw-r--r-- 1 root root    292928 Oct 27 16:54 00000000000005624695.index
-rw-r--r-- 1 root root 536882262 Oct 27 16:54 00000000000005624695.log
-rw-r--r-- 1 root root    293584 Oct 27 17:05 00000000000006562302.index
-rw-r--r-- 1 root root 536881850 Oct 27 17:05 00000000000006562302.log
-rw-r--r-- 1 root root    293424 Oct 27 17:12 00000000000007500786.index
-rw-r--r-- 1 root root 536875552 Oct 27 17:12 00000000000007500786.log
-rw-r--r-- 1 root root    292760 Oct 27 17:20 00000000000008438564.index
-rw-r--r-- 1 root root 536872517 Oct 27 17:20 00000000000008438564.log
-rw-r--r-- 1 root root    293120 Oct 27 17:31 00000000000009376250.index
-rw-r--r-- 1 root root 536886980 Oct 27 17:31 00000000000009376250.log
```

关于segment file中index与data file对应关系图，这里我们选用网上的一个图片，如下所示：

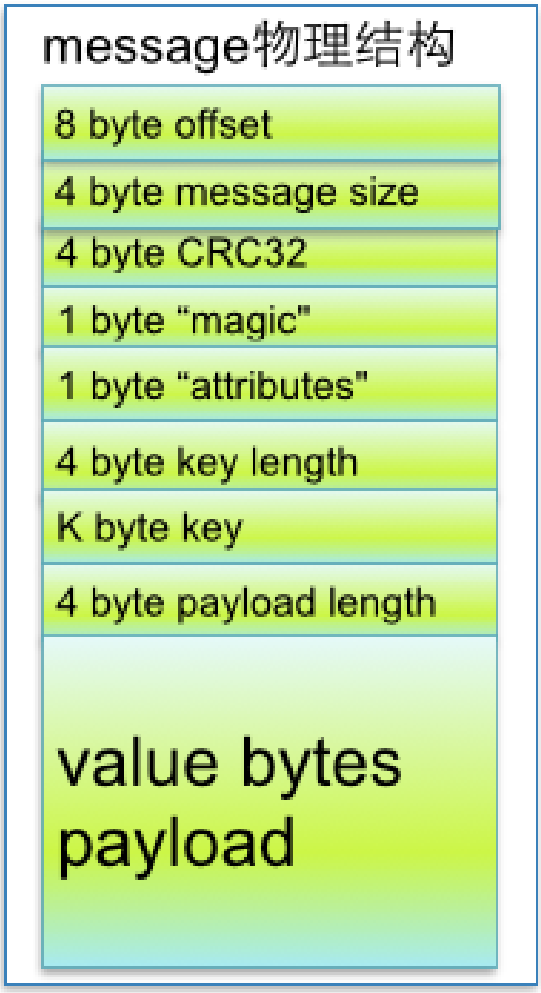


segment的索引文件中存储着大量的元数据，数据文件中存储着大量消息，索引文件中的元数据指向对应数据文件中的message的物理偏移地址。以索引文件中的 3，497 为例，在数据文件中表示第3个message（在全局partition表示第368772个message），以及该消息的物理偏移地址为497。

注：Partition中的每条message由offset来表示它在这个partition中的偏移量，这个offset并不是该Message在partition中实际存储位置，而是逻辑上的一个值（如上面的3），但它却唯一确定了partition中的一条Message（可以认为offset是partition中Message的id）。

2）message文件

message中的物理结构为：



说明如下：

关键字	解释说明
8 byte offset	在parition(分区)内的每条消息都有一个有序的id号，这个id号被称为偏移(offset),它可以唯一确定每条消息在parition(分区)内的位置。即offset表示partiion的第多少message
4 byte message size	message大小
4 byte CRC32	用crc32校验message
1 byte “magic”	表示本次发布Kafka服务程序协议版本号
1 byte “attributes”	表示为独立版本、或标识压缩类型、或编码类型
4 byte key length	表示key的长度,当key为-1时，K byte key字段不填
K byte key	可选
value bytes payload	表示实际消息数据

19、在实际的生产中，kafka常见的监控实现有哪些？（了解）

Kafka Manager：雅虎出品，可管理多个Kafka集群，是目前功能最全的管理工具。但是注意，当你的Topic太多，监控数据会占用你大量的带宽，造成你的机器负载增高。其监控功能偏弱，不满足需求。

Kafka Offset Monitor：程序一个jar包的形式运行，部署较为方便。只有监控功能，使用起来也较为安全。

Kafka Web Console：监控功能较为全面，可以预览消息，监控Offset、Lag等信息，不建议在生产环境中使用。

Burrow：是LinkedIn开源的一款专门监控consumer lag的框架。支持报警，只提供HTTP接口，没有webui。

Availability Monitor for Kafka：微软开源的Kafka可用性、延迟性的监控框架,提供JMX接口，用的很少。

Kafka Eagle：是一款结合了目前大数据Kafka监控工具的特点，重新研发的一块开源免费的Kafka集群优秀的监控工具。它可以非常方便的监控生产环境中的offset、lag变化、partition分布、owner等。在生产环境中经常使用。

另外，各种大数据管理平台比如CDH、HDP、华为的FusionInsight、星环的TDH均自带了kafka的相关监控

20、Kafkag高低级（新旧）API区别（了解）

1、高级API

优点：

高级API写起来简单，不需要去自行去管理offset，系统通过zookeeper自行管理

不需要管理分区，副本等情况，系统自动管理

消费者断线会自动根据上一次记录在 zookeeper中的offset去接着获取数据

可以使用group来区分对访问同一个topic的不同程序访问分离开来（不同的group记录不同的offset，这样不同程序读取同一个topic才不会因为offset互相影响）

缺点：

不能自行控制 offset（对于某些特殊需求来说）

不能细化控制如分区、副本、zk 等

2、低级API

优点：

能够开发者自己控制offset，想从哪里读取就从哪里读取

自行控制连接分区，对分区自定义进行负载均衡

对 zookeeper 的依赖性降低（如：offset 不一定非要靠 zk 存储，自行存储offset 即可，比如存在文件或者内存中）

缺点：

太过复杂，需要自行控制offset，连接哪个分区，找到分区leader等

21、Kafka在哪些地方会有选举过程，使用什么工具支持选举？（理解背诵）

Kafka中的选举大致可以分为三大类：**控制器的选举（先到先得）、分区leader的选举（ISR）以及消费者相关的选举**

1、控制器的选举

在Kafka集群中会有一个或多个broker，其中有一个broker会被选举为控制器（Kafka Controller），它负责管理整个集群中所有分区和副本的状态等工作。比如当某个分区的leader副本出现故障时，由控制器负责为该分区选举新的leader副本。再比如当检测到某个分区的ISR集合发生变化时，由控制器负责通知所有broker更新其元数据信息。**kafka Controller的选举是依赖Zookeeper来实现的，**在Kafka集群中哪个broker能够成功创建/controller这个临时（EPHEMERAL）节点他就可以成为Kafka Controller。

2、分区Leader的选举

分区leader副本的选举由Kafka Controller负责具体实施。当创建分区（创建主题或增加分区都有创建分区的动作）或分区上线（比如分区中原先的leader副本下线，此时分区需要选举一个新的leader上线来对外提供服务）的时候都需要执行leader的选举动作。

基本思路是按照AR（Assigned Repllicas：分区中的所有副本）集合中副本的顺序查找第一个存活的副本，并且这个副本在ISR集合中。一个分区的AR集合在分配的时候就被指定，并且只要不发生重分配的情况，集合内部副本的顺序是保持不变的，而分区的ISR集合中副本的顺序可能会改变。注意这里是根据AR的顺序而不是ISR的顺序进行选举的。这个说起来比较抽象，有兴趣的学员可以手动关闭/开启某个集群中的broker来观察一下具体的变化。

还有一些情况也会发生分区leader的选举，比如当分区进行重分配（reassign）的时候也需要执行leader的选举动作。这个思路比较简单：从重分配的AR列表中找到第一个存活的副本，且这个副本在目前的ISR列表中。

再比如当发生优先副本（preferred replica partition leader election）的选举时，直接将优先副本设置为leader即可，AR集合中的第一个副本即为优先副本。

还有一种情况就是当某节点被优雅地关闭（也就是执行ControlledShutdown）时，位于这个节点上的leader副本都会下线，所以与此对应的分区需要执行leader的选举。这里的具体思路为：从AR列表中找到第一个存活的副本，且这个副本在目前的ISR列表中，与此同时还要确保这个副本不处于正在被关闭的节点上。

3、消费者相关的选举

组协调器GroupCoordinator需要为消费组内的消费者选举出一个消费组的leader，**这个选举的算法也很简单，分两种情况分析。如果消费组内还没有leader，那么第一个加入消费组的消费者即为消费组的leader。如果某一时刻leader消费者由于某些原因退出了消费组，那么会重新选举一个新的leader，这个重新选举leader的过程又更“随意”了**，相关代码如下：

```
private val members = new mutable.HashMap[String, MemberMetadata]
var leaderId = members.keys.head
```

解释一下这2行代码：在GroupCoordinator中消费者的信息是以HashMap的形式存储的，其中key为消费者的member_id，而value是消费者相关的元数据信息。leaderId表示leader消费者的member_id，它的取值为HashMap中的第一个键值对的key，这种选举的方式基本上和随机无异。总体上来说，消费组的leader选举过程是很随意的。

用过Kafka的对partition.assignment.strategy（取值为RangeAssignor、RoundRobinAssignor、StickyAssignor等）这个参数都并不陌生。每个消费者都可以设置自己的分区分配策略，对消费组而言需要从各个消费者呈报上来的各个分配策略中选举一个彼此都“信服”的策略来进行整体上的分区分配。这个分区分配的选举并非由leader消费者决定，而是根据消费组内的各个消费者投票来决定的。

郑冬综合2502 2026/6/3 21:39:32
涤生大数据版权所有，请勿外传

无标签